Transactions
Isolation

Isolation

- DBMS can execute transactions concurrently
  - exploitation of resources
    - one transactions performs I/O, the other uses CPU etc.
    - Exploiting multi-core
- Isolation:
  - although transactions execute concurrently, each transaction runs in isolation -- not affected by the actions of other transactions
- Isolation is enforced by a **concurrency control** protocol,
- Most standard way in current DBMS to implement concurrency control is **locking**
- DBMS implements various locking strategies
  - Different locking strategies offer different *levels of isolation*
  - The strongest level ensures **serializable** executions
    - Net effect of transactions executing concurrently is identical to executing all transactions one after the other in some serial order

Outline

- "executions / schedules "
- problems that can occur when transactions are run concurrently without any concurrency control
- When is a concurrent execution equivalent to one where transactions run one after the other

Serial Execution: Example

- Consider two transactions (*Xacts*/*Txn*):

T1:
A=A+100,
B=B-100
commit

T2:
A=1.06*A,
B=1.06*B
commit

- T1 transfers $100 from B's account to A's account.
- T2 credits both accounts with a 6% interest payment.
- Assume A=B=200 at beginning
- Serial execution I: T1 executes before T2: Values of A and B?
  - A = 318, B=106 (Sum = 424)
- Serial execution II: T2 executes before T1: Values of A and B?
  - A = 312, B=112 (Sum = 424)
- Both execution orders make sense

Concurrent Execution: Example

- Consider two transactions (*Xacts*/*Txn*):

T1:
A=A+100,
B=B-100
commit

T2:
A=1.06*A,
B=1.06*B
commit

Same as:

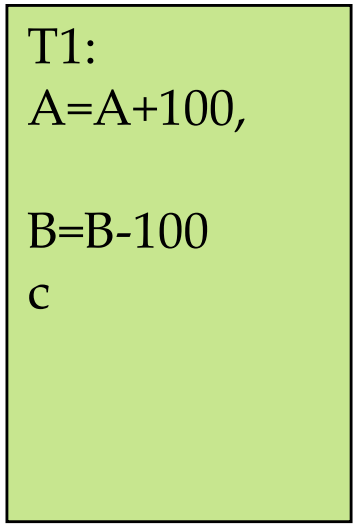
T1:
R1(A)
W1(A)
R1(B)
W1(B)
c1

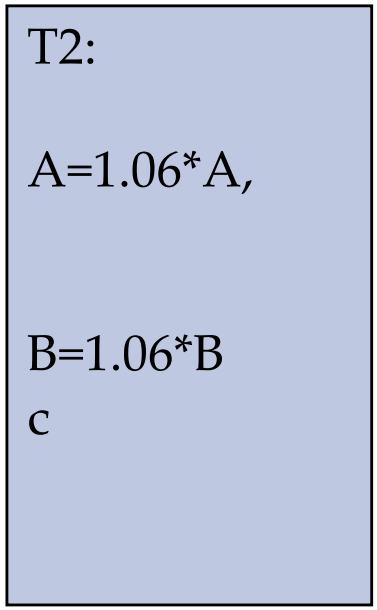
T2:
R2(A)
W2(A)
R2(B)
W2(B)
c2

- T1 transfers $100 from B's account to A's account.
- T2 credits both accounts with a 6% interest payment.
- Now assume T1 and T2 are submitted at the same time
  - No guarantee that T1 will execute before T2 or vice-versa
  - The net effect *must* be equivalent to these two transactions running serially in some order.

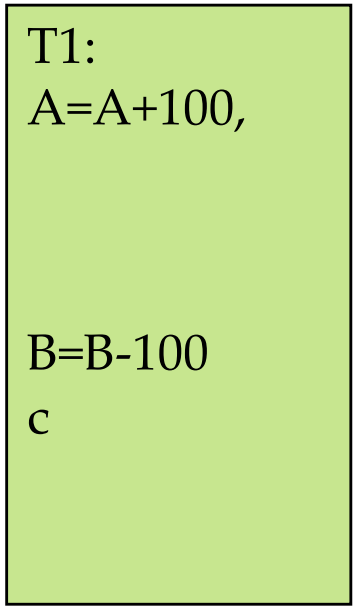
Example (Contd.)

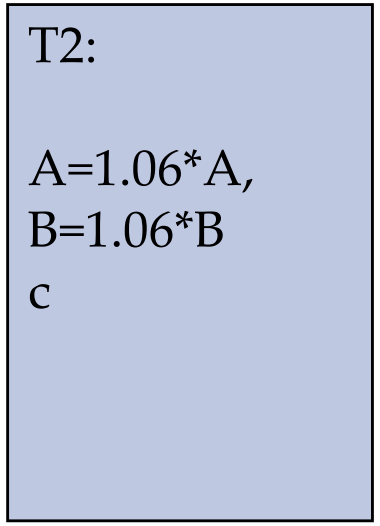
- Consider two interleavings **(schedules):**





- good





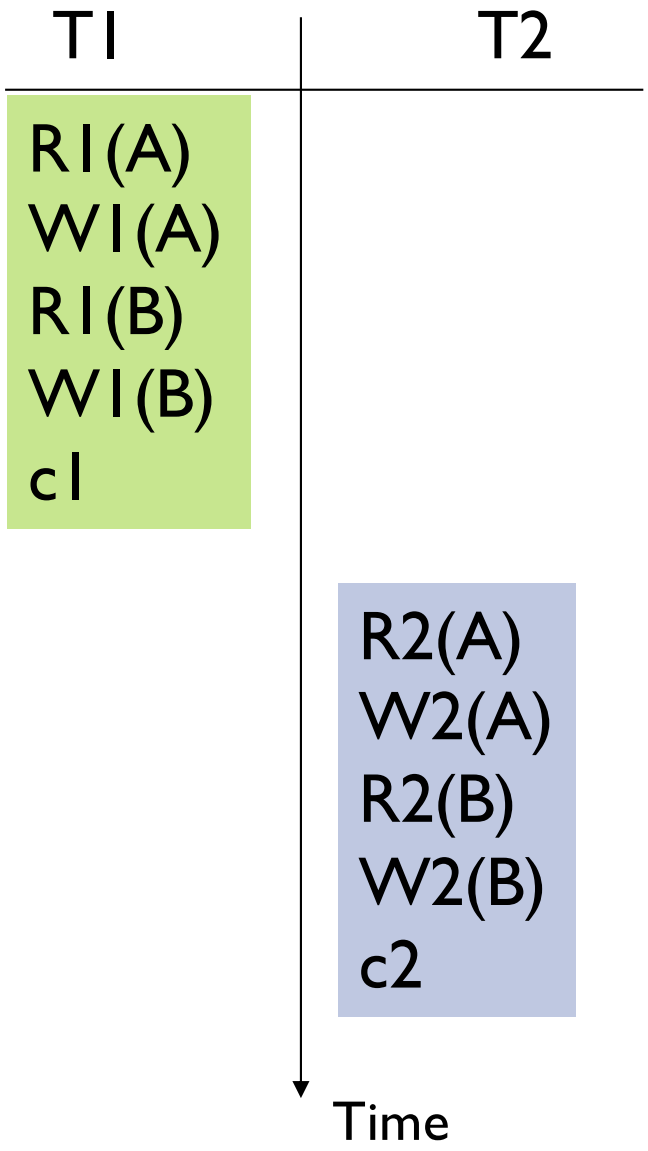
- A bad one:
  - ☆ The 100$ that are transferred are included twice in the interest rate calculation

Schedules

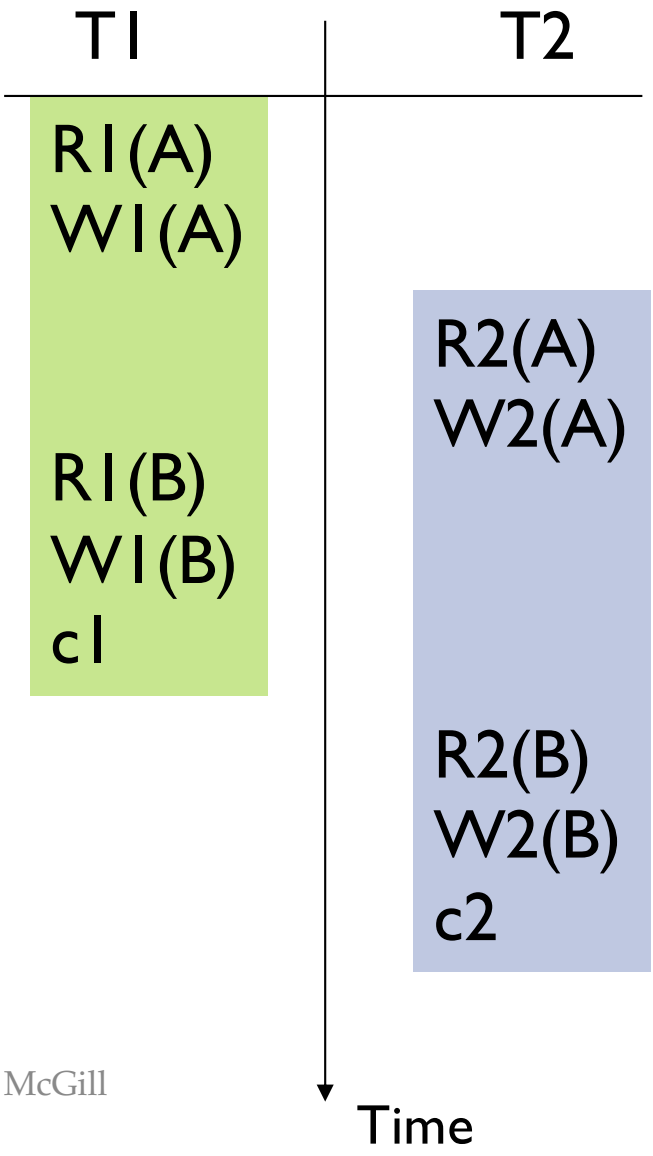
- Transaction:
  - A sequence of read and write operations on objects of the DB (denoted as r(x)/w(x))
  - Each transaction must specify as its final action either commit (c), i.e. complete successfully or abort (a), i.e., terminate and undo all the actions carried out so far.
- Schedule:
  - sequence of actions (read,write,commit,abort) from a set of transactions
  - Reflects how the DBMS sees the execution of operations; ignores things like reading/writing from OS files etc.
- Complete Schedule:
  - Contains commit/abort for each of its transactions.
- Serial schedule:
  - Schedule where transactions are executed one after the other.

Examples

Serial Schedule

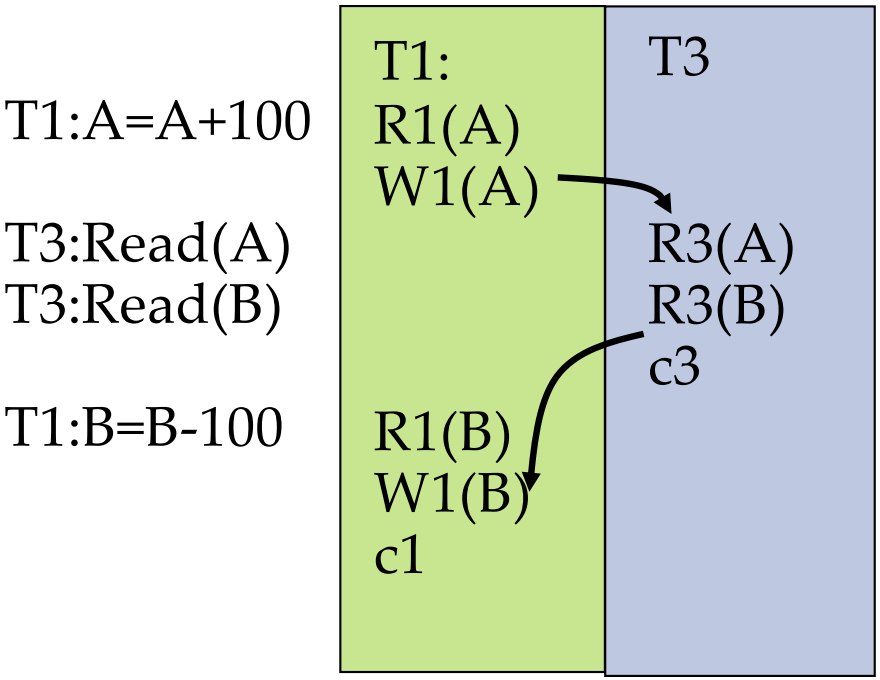


Non serial Schedule



Reading Uncommitted Data: Dirty Reads

- T1: money transfer (as before)
- T3: sum of all accounts

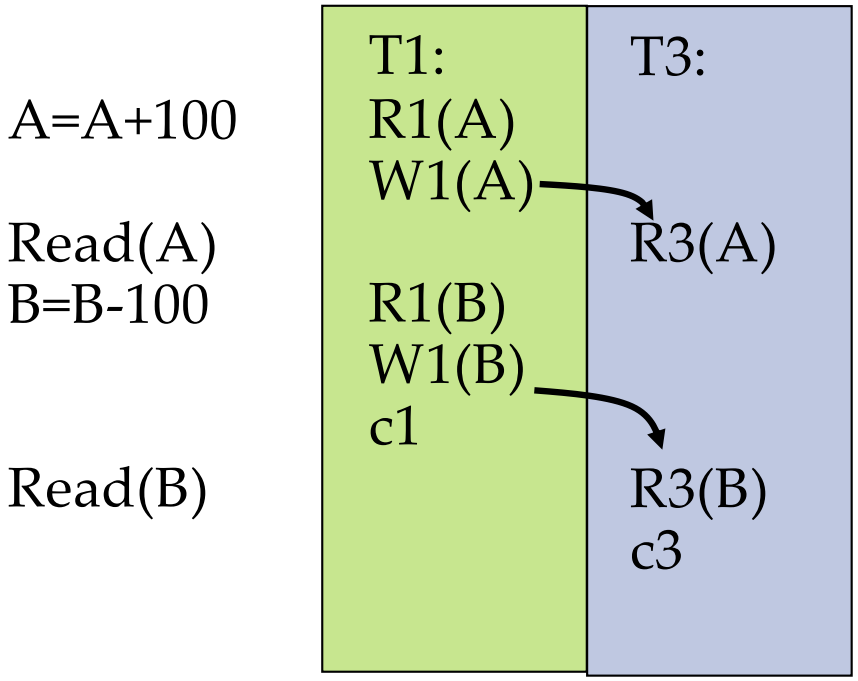


- The user perspective:
  - ☆ T3 appears to execute after T1 because it reads the A value T1 has written;
  - ☆ T3 appears to execute before T1 because it reads the B value before T1 has written

- If T3 reads from T1 before T1 commits, it might read inconsistent data (**Inconsistent or Dirty Reads**)
- Formal: a transaction Ti performs a dirty read if it reads a data item that was wrtten by a transaction Tj that has not yet committed at the time of read (i.e., wj(a), ri(a) and only sometime after cj or aj.

Reading Uncommitted Data: Dirty Reads

- T1: money transfer (as before)
- T3: sum of all accounts

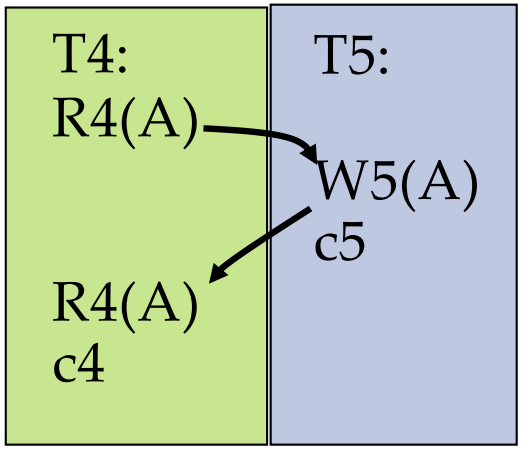


- The user perspective:
  - ☆ T3 always reads after T1 writes;
  - ☆ It is as if T3 executes serially after T1

- If T3 reads from T1 before T1 commits, it **might** read inconsistent data (**Inconsistent or Dirty Reads**)
- But it might also be ok

Unrepeatable Reads

- T4 reads A twice
- T5 updates A

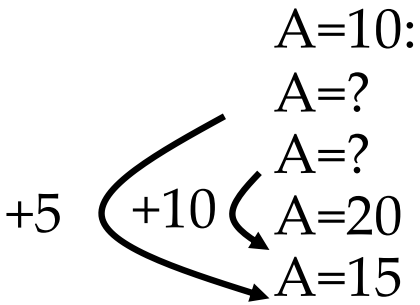


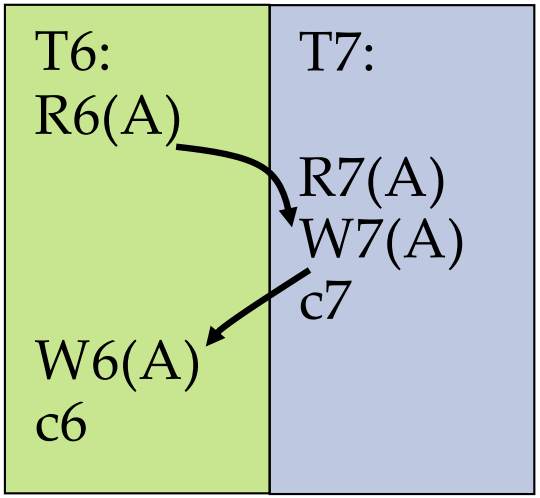
- The user perspective:
  - ☆ T4 appears to execute before T5 because it reads A before T5 writes it
  - ☆ T4 appears to execute after T5 because it reads A after T5 writes it

- Formal: If a transaction Ti reads twice the same data item and another transaction Tj changes the value between the first and second read, then we have an **unrepeatable read** situation.
- (The times of commit/abort of the two transactions do not play a role for unrepeatable reads)

Lost Update

- T6: A=A+5
- T7: A=A+10
- A=10 at start



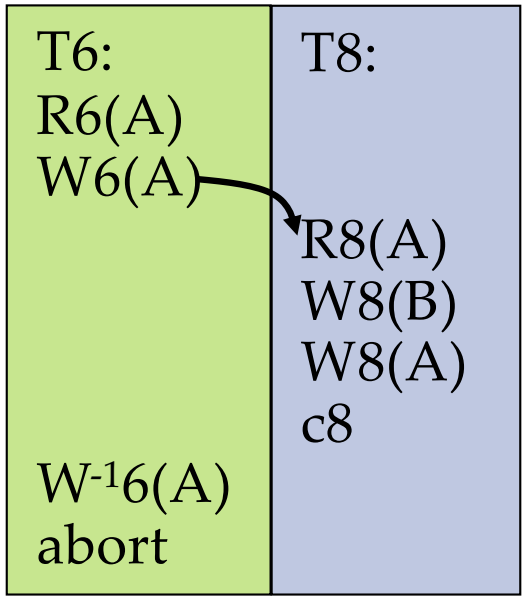


- The user perspective:
  - It is as if T7's update has never taken place; it is not reflected in the database
  - If it were reflected the final value of A should be 25 and not 15

- Can lead to **lost update**
- Formal: If transaction Ti reads a data item, then a transaction Tj writes a data item, and then Ti writes the data item (ri(a), wj(a), wi(a)), Tj's update/write is considered lost.
- (The times of commit/abort of the two transactions do not play a role for unrepeatable reads)

Committed and Aborted Transactions

- If a transaction aborts, all its actions are undone.
- It is if they were never carried out



- The user perspective:
  - T8 reads a value for A that actually will never exist!
  - Problem even bigger if the read triggered a further change in the database
  - Problem even bigger if A is updated in between; how to undo in this case???

- **Formal definition dirty write:** If a transaction Ti writes a data item and then Tj writes it before Ti commits/aborts (wi(a), wj(a), ci/ai), then Tj performs a dirty write.
- **Dirty Read** can lead to reading a non-existing value
- **Dirty Write** can mess up the database
  - AVOID AT ALL COSTS!!

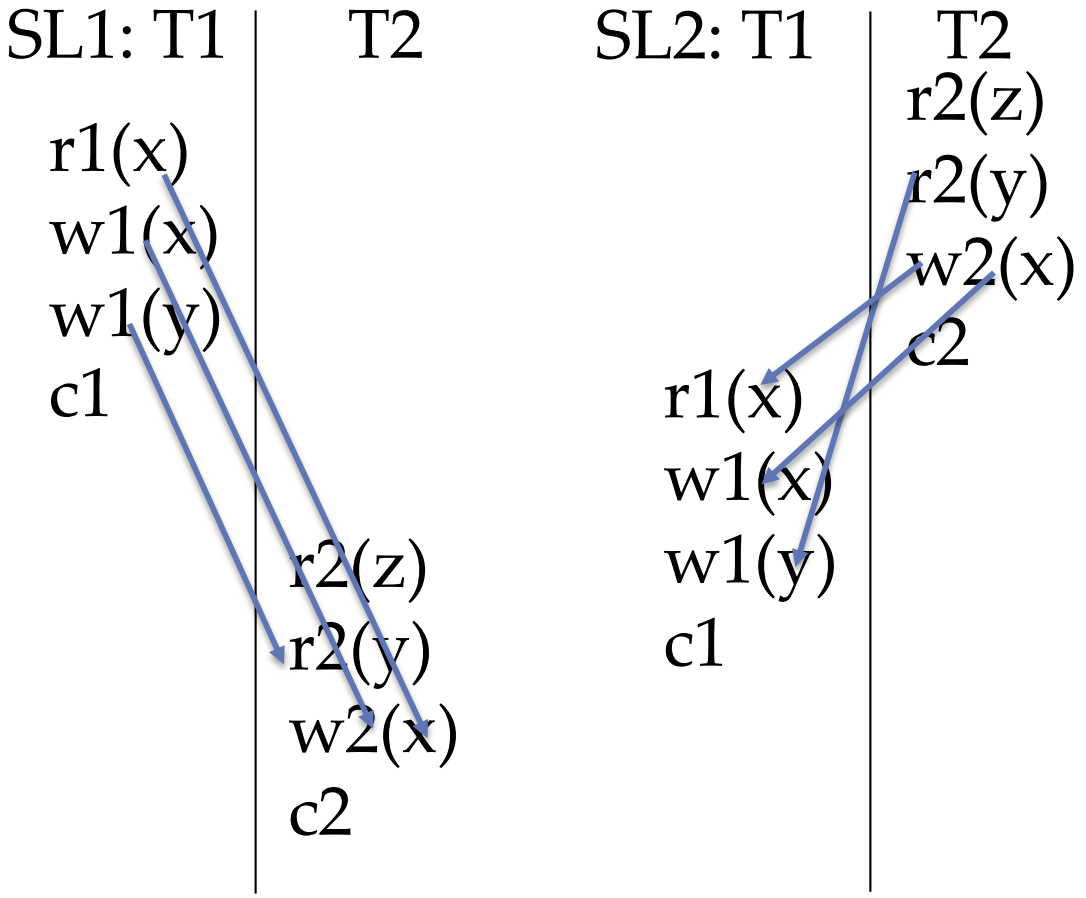
Conflicting Operations

- Conflicting operations: Two operations conflict if
  - They access the same object
  - Both operations are writes, or one is write and one is read
- Schedules
  - serial schedule: T1, T2 = r1(x),r1(y),w1(y),c1,r2(x),w2(x),r2(y),c2
    - all operations of T1 are executed before any operation of T2
    - in particular: if T1 and T2 have several conflicting operations, T1's operation is always executed before T2's conflicting operation.
  - Our problematic examples:
    - one operation of T1 was ordered before the conflicting operation of T2
    - another operation of T1 was ordered after T2's conflicting operation

Conflict Serializable Schedules

- Two schedules are conflict equivalent if:
  - Involve the same actions of the same (committed) transactions
  - Every pair of conflicting actions of (committed transactions) is ordered the same way
- Schedule S is conflict serializable if
  - S is conflict equivalent to some serial schedule which contains the committed transactions of S
  - Textbook differentiates between
    - Serializable
    - Conflict-serializable
    - View-serializable
  - Here:
    - conflict-serializable = serializable
    - Ignore view-serializable

Examples



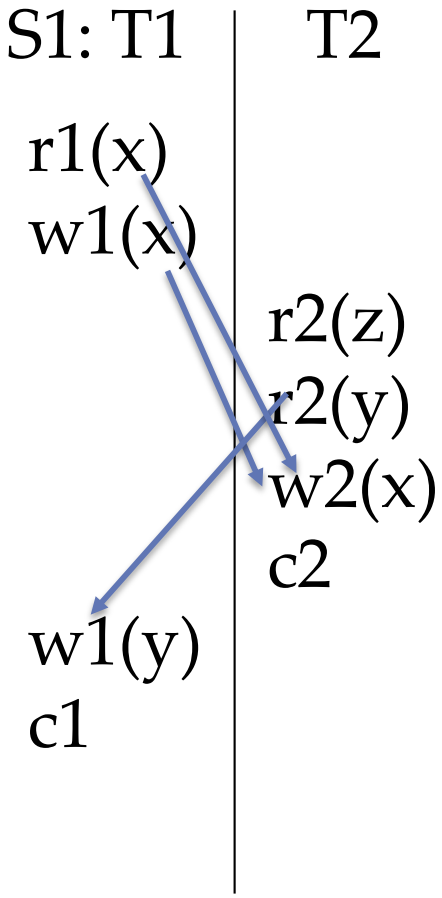
Serial Schedules

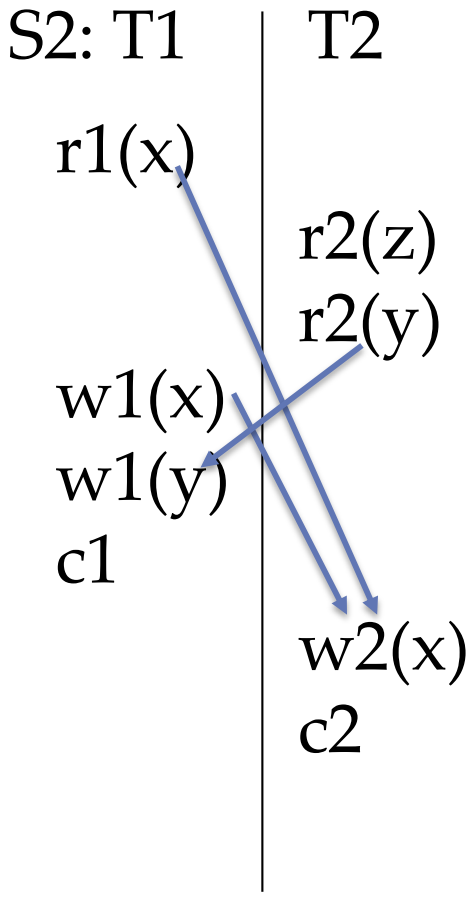
Schedules written in a different format:

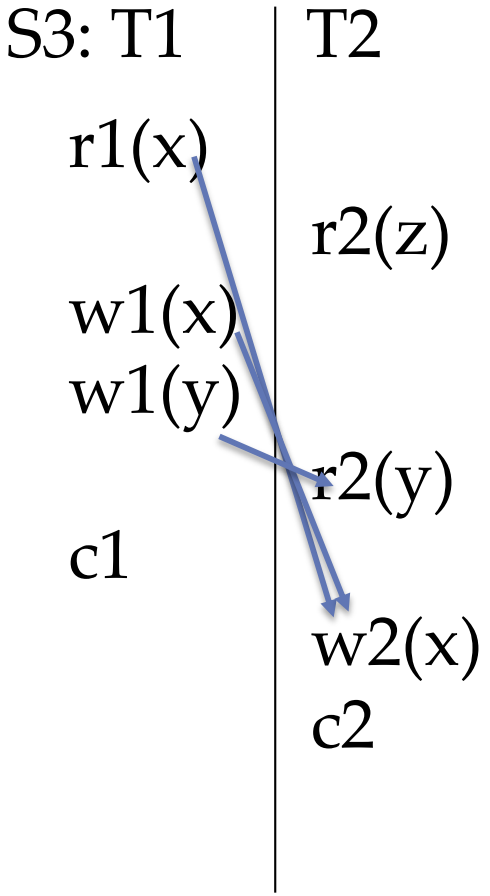
SL1: r1(x),w1(x),w1(y),c1,r2(z),r2(y),w2(x),c2. or *T1,T2*

SL2: r2(z) r2(y) w2(x) c2 r1(x) w1(x) w1(y) c1. or *T2,T1*

Examples







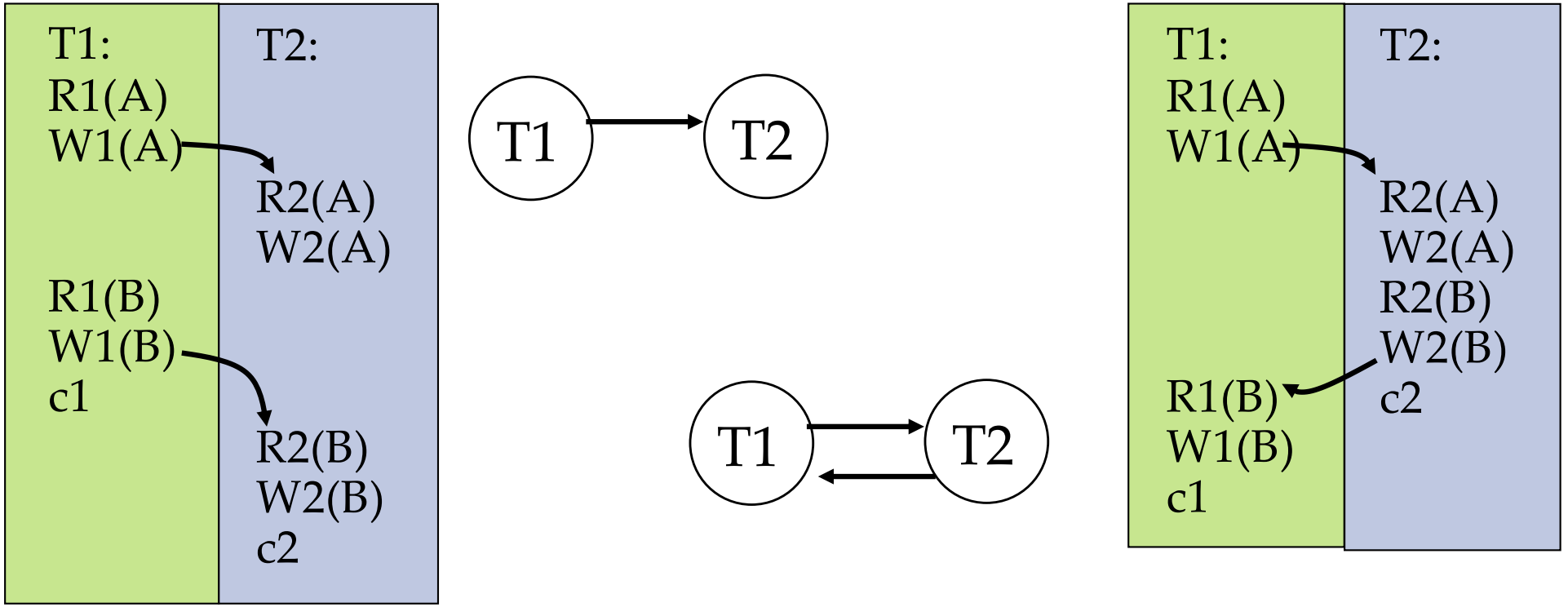
Schedules written in a different format:

S1: r1(x) w1(x) r2(z) r2(y) w2(x) c2 w1(y) c1

S2: r1(x) r2(z) r2(y) w1(x) w1(y) c1 w2(x) c2

S3: r1(x) r2(z) w1(x) w1(y) c1 r2(y) w2(x) c2

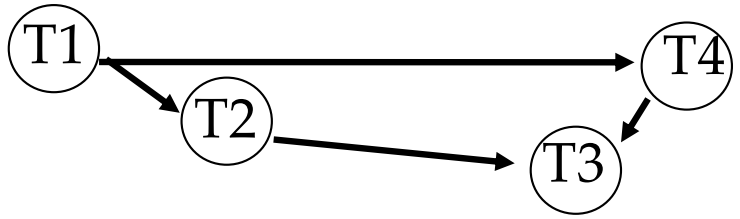
Serializability and Dependency Graphs



- Dependency graph / Serialization graph / precedence graph / Serializability graph for a schedule:
  - Let S be a schedule (T, O, <)
    - Each transaction Ti in T is represented by a node
    - There is an edge from Ti to Tj if an operations of Ti precedes and conflicts with one of Tj's operations in the schedule.

Dependency Graphs

- Theorem: Schedule is conflict serializable if and only if its dependency graph is acyclic
- Generating an equivalent serial schedule
  - Continue until no nodes are left
    - Choose a source (i.e. a node without incoming edges)
      - put the corresponding transaction next in the serial order
      - Delete the node and all outgoing edges



T1

T1 -> T2

T1 -> T2 -> T4

T1 -> T2 -> T4 -> T3

T1 -> T4 -> T2 -> T3